Chapitre 9 : Algorithmes gloutons

ITC - 2025/2026

Félix POTIÉ

Sommaire

Sommaire 2
Problème d'optimisation 3
Exemple : le problème du voyageur 4
Algorithmes gloutons 8
Optimalité 15

Problème d'optimisation

Définition 1: Problème d'optimisation.

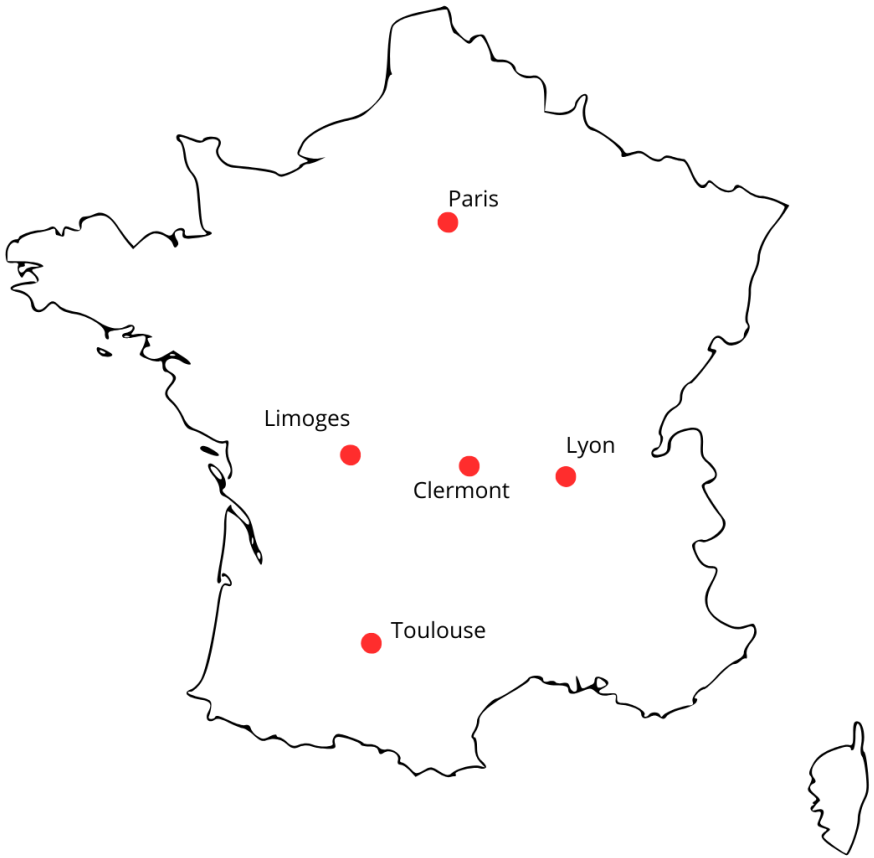
Un **problème d'optimisation** est un problème algorithmique dans lequel l'objectif est de trouver la « **meilleure** » solution (selon un **critère** donné) parmi un ensemble de solutions également valides mais potentiellement moins bonnes.

Le contexte d'un problème d'optimisation est donc :

- un très **grand nombre de solutions** (dans le cas contraire, il n'y aurait pas de difficulté à trouver la meilleure)
- une fonction permettant d'**évaluer la qualité** de chaque solution
- l'existence d'une **solution optimale**, ou suffisamment bonne.

Exemple : le problème du voyageur

Problème : Un voyageur souhaite visiter plusieurs villes de France, dans n'importe quel ordre, mais en minimisant la distance parcourue.



Exemple : le problème du voyageur - 2

- Départ : *Clermont-Ferrand*
- Villes à visiter : *Limoges, Lyon, Paris et Toulouse*

	Clermont-Ferrand	La Rochelle	Lille	Limoges	Lyon	Marseille	Nantes	Paris
La Rochelle	462							
Lille	622	688						
Limoges	173	222	608					
Lyon	165	614	681	385				
Marseille	413	823	1001	593	314			
Nantes	534	137	600	320	655	986		
Paris	423	472	225	392	466	775	385	
Toulouse	377	421	894	290	467	404	585	678

Exemple : le problème du voyageur - 3

On peut calculer les distances de **toutes** les possibilités pour trouver l'**optimal** :

Trajet (départ/et arrivée à Clermont Ferrand)	**Distance** (km)
Limoges – Lyon – Paris – Toulouse	2079
Limoges – Lyon – Toulouse – Paris	2126
Limoges – Paris – Lyon – Toulouse	1875
Limoges – Paris – Toulouse – Lyon	1875
Limoges – Toulouse – Lyon – Paris	1819
Limoges – Toulouse – Paris – Lyon	1772
Lyon – Limoges – Paris – Toulouse	1997
Lyon – Limoges – Toulouse – Paris	1941
Lyon – Paris – Limoges – Toulouse	**1690**
Lyon – Toulouse – Limoges – Paris	1737
Paris – Limoges – Lyon – Toulouse	2044
Paris – Lyon – Limoges – Toulouse	1941

Exemple : le problème du voyageur - 4

Énumération de toutes les solutions :

- souvent peu pertinente (temps de calculs trop longs)

L'ensemble des trajets possibles correspond à la moitié de l'ensemble des permutations possibles des n villes à visiter, soit $\frac{n!}{2}$.

- Avec 4 villes : 12 solutions
- Avec 8 villes : 20160 solutions
- Avec 20 villes : $1,2 \cdot 10^{18}$ solutions, *ce qui représente plusieurs jours de calculs*

On va donc chercher des solutions plus efficaces de trouver la solution optimale **ou une solution qui s'en rapproche**.

Algorithmes gloutons

Les algorithmes gloutons :

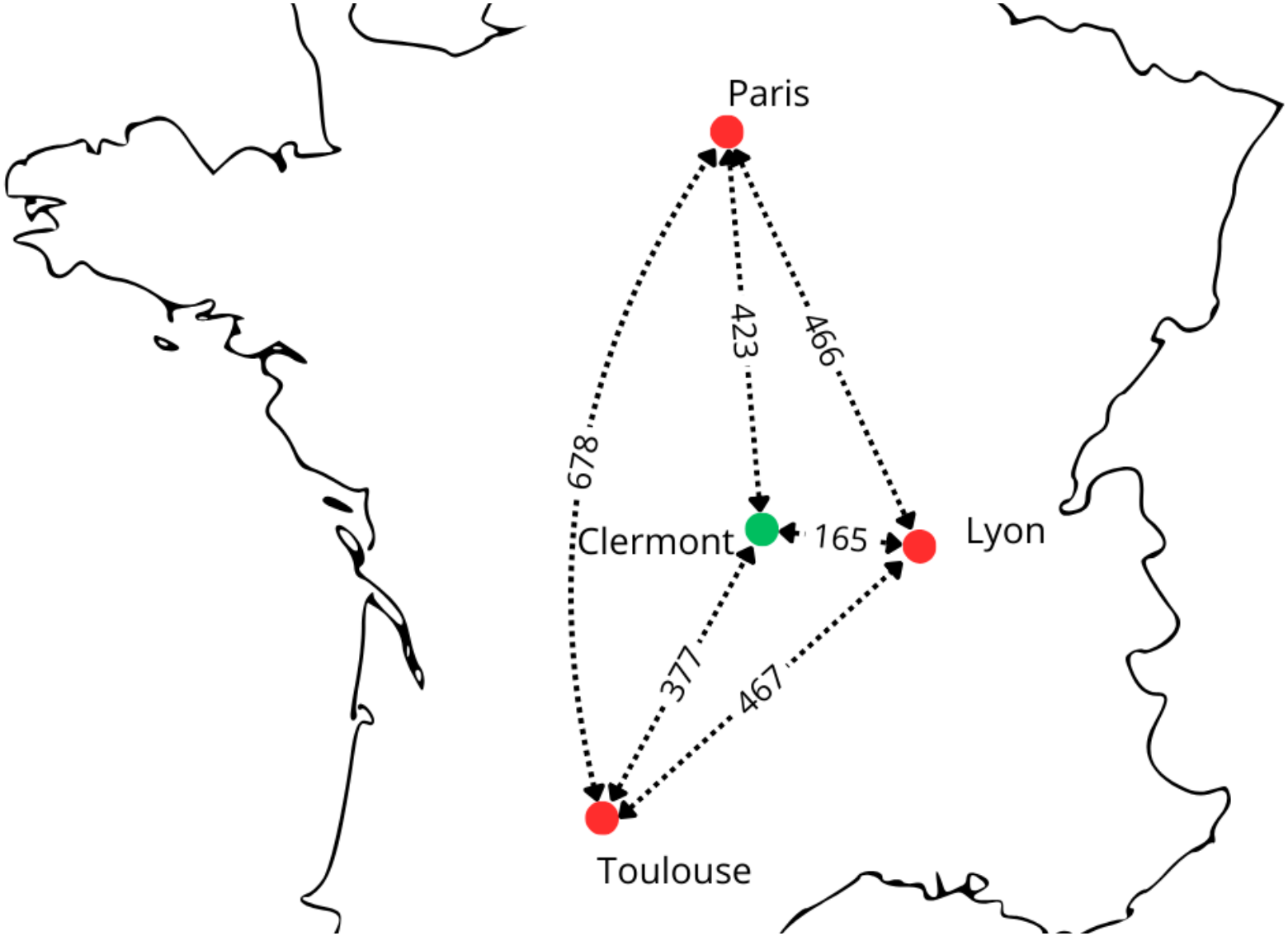
- beaucoup plus simple à programmer
- mais dont le résultat **n'est pas toujours optimal**.

On cherche une solution **suffisamment bonne** : proche de l'optimum, obtenue rapidement.

Définition 2: Algorithme glouton.

L'approche gloutonne consiste à construire une solution complète par une **succession de choix locaux** donnant systématiquement **la meilleure solution partielle**.

Algorithmes gloutons - 2



Algorithmes gloutons - 3

Dans la suite de ce cours, on simplifie le problème en enlevant le fait de devoir revenir sur la ville de départ.

Stratégie gloutonne 1 :

- On prend la ville restante la plus proche qu'on n'a pas encore visitée
- On recommence jusqu'à avoir visité toutes les villes

Ici on ferait :

$$\text{Clermont} \xrightarrow{165} \text{Lyon} \xrightarrow{466} \text{Paris} \xrightarrow{678} \text{Toulouse} = 1\ 309$$

Question : Est-ce que cette stratégie est optimale ?

Algorithmes gloutons - 4

Cette stratégie **n'est pas optimale** pour toutes les configurations. Par exemple si Clemont→Toulouse serait à 166km, on prendrait toujours :

$$\text{Clermont} \xrightarrow{165} \text{Lyon} \xrightarrow{466} \text{Paris} \xrightarrow{678} \text{Toulouse} = 1\ 309$$

alors qu'on pourrait avoir moins en faisant :

$$\text{Clermont} \xrightarrow{166} \text{Toulouse} \xrightarrow{467} \text{Lyon} \xrightarrow{466} \text{Paris} = \mathbf{1\ 099}$$

Algorithmes gloutons - 5

Stratégie gloutonne 2 :

- On choisi une ville pas encore visité (aléatoirement)
- On regarde à quel emplacement l'ajouter dans notre trajet de façon à minimiser la longueur du trajet.
- On recommence jusqu'à avoir visité toutes les villes

Remarque : on a donc plusieurs réponse possible ici

Algorithmes gloutons - 6

Si on prend les villes dans l'ordre Clermont-Lyon-Paris-Toulouse :

- Tour 1 : On ajoute Lyon :
  Clermont→Lyon = **165**
- Tour 2 : On ajoute Paris, on compare:
  Clermont→Lyon→Paris = **631** et
  Clermont→Paris→Lyon = 889
- Tour 3 : On ajoute Toulouse, on compare:
  Clermont→Toulouse→Lyon→Paris = 1 310 et
  Clermont→Lyon→Toulouse→Paris = 1 310 et
  Clermont→Lyon→Paris→Toulouse = **1 309**

Algorithmes gloutons - 7

Si on prend les villes dans l'ordre Clermont-Toulouse-Paris- Lyon :

- Tour 1 : On ajoute Toulouse :
  Clermont→Toulouse = **377**
- Tour 2 : On ajoute Paris, on compare:
  Clermont→Paris→Toulouse = 1 101 et
  Clermont→Toulouse→Paris = **1 055**
- Tour 3 : On ajoute Lyon, on compare:
  Clermont→Lyon→Toulouse→Paris = **1 310** et
  Clermont→Toulouse→Lyon→Paris = **1 310** et
  Clermont→Toulouse→Paris→Lyon = 1 521

Ici on n'arrive pas sur le résultat optimal. Donc notre stratégie gloutonne n'est toujours pas optimal.

Optimalité

Comment savoir si un algorithme glouton est optimal ?

C'est compliqué !

Pour la non-optimalité, il faut réfléchir à des cas différents, qui changerait le fonctionnement de l'algorithme.

Optimalité - 2

Ce qui vous est demandé en ITC :

- connaitre le principe des algorithmes gloutons
- savoir qu'ils ne sont pas toujours optimaux
- dans certains cas sur des exemples connues (qu'on verra en TP) de montrer la **non optimalité**
- on ne vous demandera pas de prouver l'optimalité d'un algorithme glouton

Les TP permettront d'illustrer deux situations opposées :

- un problème où une stratégie gloutonne est toujours optimale
- un problème où aucune stratégie gloutonne simple ne fonctionne